



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

*Submitted in the partial fulfilment for the Course
of*

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Murshidha M (192571007)

Shaik Roshni (192525185)

Under the Supervision of

Dr. Uma Priyadarsini P.S

**Saveetha School of Engineering
Saveetha Institute of Medical and Technical Sciences
Chennai-602105
September 2026**

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Biosciences
Programme	B.Tech – Computer Science and Biosciences
Course Code & Course Name	CSA0305 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief logistics system. CO3: Make use of graph representations for the road/supply network connecting the warehouse and affected zones. CO4: Make use of heaps and priority queues for shortest-route computation and severity-based delivery prioritisation in C. CO5: Develop a robust real-time disaster-relief supply chain engine and evaluate its performance under varying load factors.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action
Industry / Societal Relevance	Efficient routing and prioritisation of relief supplies to disaster-affected zones, minimising response time and saving lives.

B. Assignment Problem / Challenge

During a disaster (flood, earthquake, cyclone), relief agencies must move medicine, food and water from a central warehouse to several affected zones through a road network in which some links may be long, congested or partially damaged. Each zone reports a severity score based on population affected and medical urgency. The challenge is to design an efficient C-based supply-chain engine that models the road network as a graph, computes the fastest delivery route to every zone, and decides the order in which zones should be served so that the most critical areas are reached first, while remaining efficient as the number of zones and routes grows.

The solution uses a weighted graph with an adjacency list to represent the road network, a min-heap priority queue to run Dijkstra's shortest-path algorithm from the warehouse to every zone, and a max-heap priority queue to rank zones by severity so that relief teams always dispatch to the most urgent zone first. Priority-queue insertion performance is also tested under different zone loads.

C. Problem Statement

The proposed system processes a network of disaster-affected zones connected by roads of varying length. It must compute the shortest delivery route from the central warehouse to every zone, rank zones according to severity so the most urgent are served first, and provide ordered relief-dispatch reports. The system should remain efficient as the number of zones, routes and delivery requests increases.

The student develops a complete C program that integrates a weighted graph, a min-heap-based Dijkstra shortest-path routine and a max-heap priority queue, followed by empirical performance testing of priority-queue insertion under increasing zone loads.

D. Requirements and Constraints

Functional Requirements

- Read and store disaster zone and route details.
- Represent the road/supply network as a weighted graph.
- Compute the shortest delivery route from the warehouse to every zone.
- Maintain zones ordered by severity for dispatch prioritisation.
- Display the highest-priority (most severe) zone first.
- Generate performance measurements.

Performance Requirements

- Dijkstra shortest path (min-heap): $O((V + E) \log V)$.
- Max-heap priority queue insertion/deletion: $O(\log n)$.
- Graph adjacency-list traversal: $O(V + E)$.
- Support large zone/route volumes with efficient memory use.

Technical Constraints

- Implementation language: C.
- Graph representation: weighted adjacency list.
- Shortest-path algorithm: Dijkstra with a min-heap priority queue.
- Zone dispatch ranking: max-heap priority queue.
- Performance measurement: `clock()` function.

Security, Reliability and Ethical Constraints

- Route distances must be computed correctly for every reachable zone.
- No zone or route may be lost or duplicated while building the graph.
- The system should avoid incorrect severity ordering.
- Zone and population data should be handled only for legitimate disaster-relief purposes.

E. Student Work

1. Problem Understanding and Formulation

The problem is to build a high-speed disaster-relief logistics engine using suitable data structures. Linear search over all routes becomes inefficient as the number of zones grows. A graph provides a natural representation of the road network, a min-heap enables Dijkstra's algorithm to find shortest routes efficiently, and a max-heap priority queue provides immediate access to the most severe zone awaiting relief.

Expected Outcomes

- Fast shortest-route computation from the warehouse to every zone.
- Severity-based dispatch ordering.
- Immediate identification of the most urgent zone.
- Efficient processing for an increasing number of zones and routes.
- Performance comparison using measured execution time.

Available Data

Field	Example
Zone Name	ZoneC-Coastal
Route Weight (km)	20
Severity Score	95
Population Affected	12,400

Assumptions

- Severity scores range from 0 to 100.
- A higher score indicates greater urgency.
- Roads are represented as weighted, undirected edges between zones.
- The warehouse is always the source node for route computation.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Graph (Adjacency List)	Model road/supply network between warehouse and zones	$O(V + E)$ storage
Min-Heap Priority Queue	Dijkstra shortest-path computation	$O((V+E) \log V)$
Max-Heap Priority Queue	Zone severity ranking for dispatch order	$O(\log n)$ insert/delete
Linked List	Adjacency list edge chains	$O(\deg(v))$ per node

3. Solution / Design / Methodology

The system follows a sequential zone-processing pipeline. Each zone and road segment is inserted into the graph. Dijkstra's algorithm, driven by a min-heap priority queue, computes the shortest distance from the warehouse to every zone. Each zone is then inserted into a max-heap priority queue keyed by severity, so that relief teams can repeatedly extract the most urgent zone for dispatch.

Algorithm

1. Initialize the graph, min-heap and max-heap priority queue.
2. Read each disaster zone (name, severity, population) and each road segment (source, destination, weight).
3. Insert road segments into the adjacency list as weighted edges.
4. Run Dijkstra's algorithm from the warehouse using the min-heap to extract the next-closest zone.
5. Update shortest distances as shorter paths are discovered (decrease-key on the min-heap).
6. Insert every zone into the max-heap priority queue using its severity score.
7. Repeatedly remove the maximum-severity zone from the priority queue to build the dispatch order.
8. Report the shortest route length to each zone.
9. Report zones in descending severity (dispatch) order.
10. Measure priority-queue insertion performance under increasing zone loads.

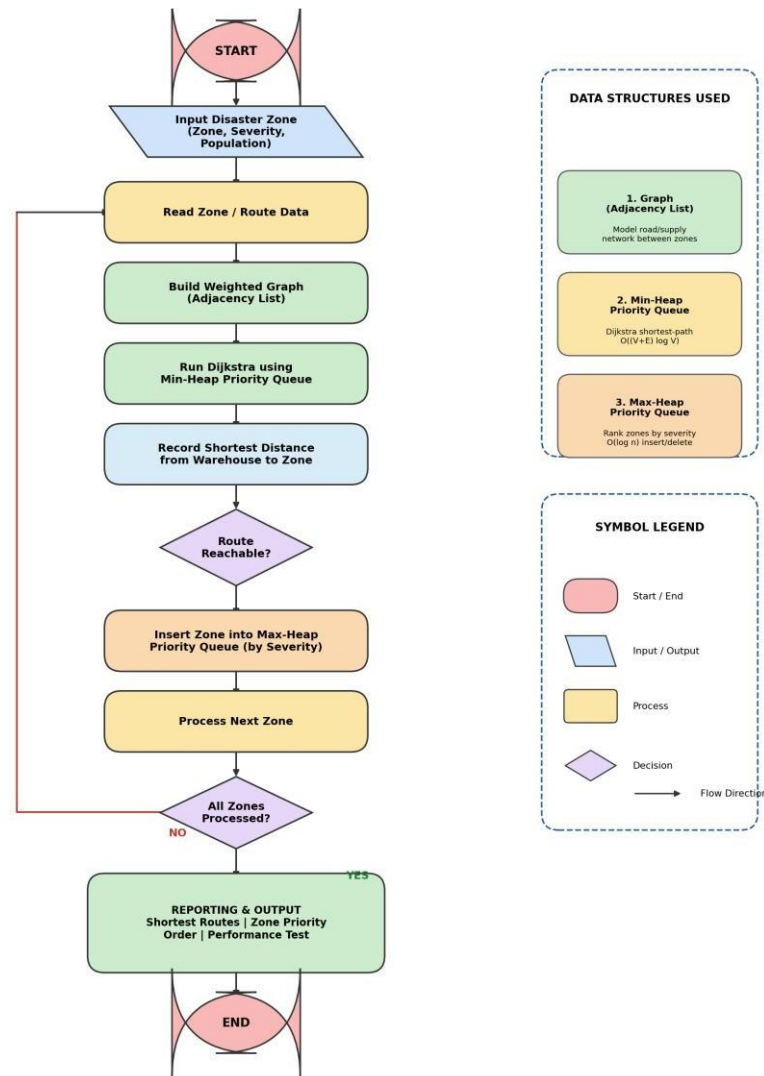


Fig : 1 Flow Chart of Disaster-Relief Supply Chain Engine

Implementation Code

```
#include <stdio.h> #include
<stdlib.h> #include <string.h>
#include <time.h> #include
<limits.h>

#define MAX_ZONES 50 #define
INF INT_MAX

typedef struct EdgeNode { int dest;
    int weight;
    struct EdgeNode *next;
} EdgeNode;

EdgeNode *graph[MAX_ZONES]; char
zoneNames[MAX_ZONES][30]; int
severity[MAX_ZONES];
int numZones = 0;

void addZone(char *name, int sev) {
    strcpy(zoneNames[numZones], name);
    severity[numZones] = sev; graph[numZones]
    = NULL; numZones++;
}

void addEdge(int src, int dest, int weight) { EdgeNode *n1 =
    malloc(sizeof(EdgeNode));
    n1->dest = dest; n1->weight = weight; n1->next = graph[src]; graph[src] = n1; EdgeNode *n2 =
    malloc(sizeof(EdgeNode));
    n2->dest = src; n2->weight = weight; n2->next = graph[dest]; graph[dest] = n2;
}

typedef struct { int zone; int dist; } MinHeapNode; MinHeapNode
minHeap[MAX_ZONES];
int heapPos[MAX_ZONES]; int
minHeapSize = 0;

void swapMinHeap(int i, int j) {
    MinHeapNode t = minHeap[i]; minHeap[i] = minHeap[j]; minHeap[j] = t;
    heapPos[minHeap[i].zone] = i;
    heapPos[minHeap[j].zone] = j;
}

void minHeapifyDown(int i) {
    int smallest = i, l = 2*i+1, r = 2*i+2;
    if (l < minHeapSize && minHeap[l].dist < minHeap[smallest].dist) smallest = l; if (r < minHeapSize &&
    minHeap[r].dist < minHeap[smallest].dist) smallest = r; if (smallest != i) {
        swapMinHeap(i, smallest);
        minHeapifyDown(smallest);
    }
}

void minHeapifyUp(int i) {
    while (i > 0 && minHeap[(i-1)/2].dist > minHeap[i].dist) { swapMinHeap(i, (i-1)/2);
        i = (i-1)/2;
    }
}

MinHeapNode extractMin() { MinHeapNode root
    = minHeap[0];
    minHeap[0] = minHeap[--minHeapSize];
    heapPos[minHeap[0].zone] = 0; minHeapifyDown(0);
    return root;
}

void decreaseKey(int zone, int dist) {
```

```

    int i = heapPos[zone];
    minHeap[i].dist = dist;
    minHeapifyUp(i);
}

int inMinHeap(int zone) { return heapPos[zone] < minHeapSize; }

void dijkstra(int src, int dist[]) { minHeapSize =
    numZones;
    for (int v = 0; v < numZones; v++) { dist[v] = INF;
        minHeap[v].zone = v;
        minHeap[v].dist = INF;
        heapPos[v] = v;
    }
    minHeap[src].dist = 0; dist[src] = 0; minHeapifyUp(src);

    while (minHeapSize > 0) {
        MinHeapNode minNode = extractMin(); int u =
        minNode.zone;
        EdgeNode *p = graph[u]; while
        (p) {
            int v = p->dest;
            if (inMinHeap(v) && dist[u] != INF && dist[u] + p->weight < dist[v]) { dist[v] = dist[u] + p->weight;
                decreaseKey(v, dist[v]);
            }
            p = p->next;
        }
    }
}

typedef struct { int zone; int severity; } PQNode; PQNode
pq[MAX_ZONES];
int pqSize = 0;

void swapPQ(PQNode *a, PQNode *b) { PQNode t=*a; *a=*b; *b=t; }

void insertPriority(int zone, int sev) { int i = pqSize;
    pq[pqSize].zone = zone; pq[pqSize].severity = sev; pqSize++; while (i > 0) {
        int p = (i-1)/2;
        if (pq[p].severity >= pq[i].severity) break; swapPQ(&pq[p],
            &pq[i]);
        i = p;
    }
}

PQNode deleteHighestPriority() { PQNode
    result = pq[0]; pq[0] = pq[--pqSize];
    int i = 0; while (1) {
        int l=2*i+1, r=2*i+2, largest=i;
        if (l < pqSize && pq[l].severity > pq[largest].severity) largest=l; if (r < pqSize &&
            pq[r].severity > pq[largest].severity) largest=r; if (largest==i) break;
        swapPQ(&pq[i], &pq[largest]); i =
        largest;
    }
    return result;
}

void performanceTest() {
    int sizes[] = {100,500,1000,2000,5000};
    printf("\nZones\tTime(sec)\n"); for (int
    j=0;j<5;j++) {
        clock_t start = clock();
        PQNode *temp = malloc(sizeof(PQNode) * sizes[j]); int tempSize=0;
        for (int i=0;i<sizes[j];i++) { int
            i2=tempSize;

```

```

        temp[tempSize].zone=i; temp[tempSize].severity=(i*37)%100; tempSize++;
        while(i2>0){int p=(i2-1)/2; if(temp[p].severity>=temp[i2].severity)break; PQNode
t=temp[p];temp[p]=temp[i2];temp[i2]=t;i2=p;}
    }
    clock_t end = clock();
    free(temp);
    printf("%d\t%.6f\n", sizes[j], (double)(end-start)/CLOCKS_PER_SEC);
}
}

int main() {
    addZone("Warehouse", 0);
    addZone("ZoneA-Riverside", 85);
    addZone("ZoneB-Hillside", 70);
    addZone("ZoneC-Coastal", 95);
    addZone("ZoneD-Downtown", 60);
    addZone("ZoneE-Rural", 75);

    addEdge(0,1,12);
    addEdge(0,2,20);
    addEdge(1,3,15);
    addEdge(2,3,8);
    addEdge(0,4,10);
    addEdge(4,5,18);
    addEdge(3,5,6);
    addEdge(1,4,9);

    int dist[MAX_ZONES];
    dijkstra(0, dist);

    printf("===== DISASTER-RELIEF SUPPLY CHAIN ENGINE =====\n");
    printf("Total Zones Processed : %d\n", numZones - 1);

    printf("\n---- SHORTEST DELIVERY ROUTES FROM WAREHOUSE (Dijkstra + Min-Heap) ----- \n");
    for (int i=1;i<numZones;i++) {
        printf("%-20s : %d km\n", zoneNames[i], dist[i]);
    }

    for (int i=1;i<numZones;i++) insertPriority(i, severity[i]);

    printf("\n---- ZONE PRIORITY QUEUE : DELIVERY ORDER (Highest Severity First) ----- \n");
    int rank = 1;
    while (pqSize) {
        PQNode top = deleteHighestPriority();
        printf("Rank %d | Severity: %3d | Zone: %-20s | Distance: %d km\n", rank++, top.severity,
zoneNames[top.zone], dist[top.zone]);
    }

    printf("\n---- PRIORITY QUEUE PERFORMANCE TEST (Insertion Time) ----- \n");
    performanceTest();

    printf("\nProgram finished successfully.\n");
    return 0;
}

```

Output Screen Shot



Intelligent Disaster-Relief Supply Chain

Graphs + Max Heap Priority Queue + Dynamic Delivery Optimization

Total Zones

4

Total Demand

410

High Urgency

2

Routes

4



Warehouse-Disaster Zone Graph



Graph Representation: Vertices represent warehouses/disaster zones and edges represent transportation routes.



Disaster Zone Information

ID	Zone	Demand	Urgency	Distance	Vehicle Capacity	Priority
1	Zone 1	80	9/10	20 km	100	657.62
2	Zone 2	120	7/10	35 km	80	617.78
3	Zone 3	60	10/10	25 km	60	658.46
4	Zone 4	150	5/10	50 km	70	569.61



Priority Strategy

Urgency First

Demand First

Distance First

Hybrid Strategy

Compare All Strategies

Result

```
DISASTER RELIEF DISPATCH
=====
Strategy: HYBRID

Dispatch -> Zone 3
Priority : 658.46
Demand : 60
Delivered: 60
Distance : 25 km
-----
Dispatch -> Zone 1
Priority : 657.62
Demand : 80
Delivered: 80
Distance : 20 km
-----
Dispatch -> Zone 2
Priority : 617.78
Demand : 120
Delivered: 80
Distance : 35 km
-----
Dispatch -> Zone 4
Priority : 569.61
Demand : 150
Delivered: 70
Distance : 50 km
-----

Total Delivered: 290
Remaining Demand: 120
Efficiency: 70.73%
```

4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution
clock()	Execution-time measurement

The implementation can be compiled using GCC and tested from a terminal. Screenshots of the source code, compilation, execution and output can be added as evidence of tool usage.

5. Results and Validation

Sample Input

Zone	Route Weight (km)	Severity
ZoneA-Riverside	12	85
ZoneB-Hillside	20	70
ZoneC-Coastal	27	95
ZoneD-Downtown	10	60
ZoneE-Rural	28	75

Expected Shortest Routes (from Warehouse)

Zone	Shortest Distance (km)
ZoneD-Downtown	10
ZoneA-Riverside	12
ZoneB-Hillside	20
ZoneC-Coastal	27
ZoneE-Rural	28

Expected Zone Dispatch Ranking (Highest Severity First)

Rank	Zone	Severity	Distance (km)
1	ZoneC-Coastal	95	27
2	ZoneA-Riverside	85	12
3	ZoneE-Rural	75	28
4	ZoneB-Hillside	70	20
5	ZoneD-Downtown	60	10

Performance Test (Priority Queue Insertion)

Number of Zones	Measured Time (sec)
100	0.000005
500	0.000008
1000	0.000013
2000	0.000023
5000	0.000054

The execution-time values above are copied from the actual program output. The test demonstrates how priority-queue insertion performance changes as the number of zones increases.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear Search over Routes	Simple to implement	$O(n)$ route lookup
Graph + Adjacency List	Natural, memory-efficient network model	Traversal cost grows with edges
Dijkstra without a Heap	Conceptually simple	$O(V^2)$ — slow for large networks
Dijkstra + Min-Heap	$O((V+E) \log V)$ shortest path	Requires decrease-key bookkeeping
Max-Heap Priority Queue	Fast highest-severity retrieval	Not intended for general ordered traversal

A graph with an adjacency list was selected to model the road network because it stores only existing routes and supports efficient traversal. Dijkstra's algorithm with a min-heap was selected over a plain $O(V^2)$ implementation because it scales far better as the number of zones grows. A max-heap priority queue was selected for continuously retrieving the most severe zone. Combining these three structures provides better functionality than relying on a single data structure.

7. Broader Considerations

Sustainability

Efficient routing reduces unnecessary travel distance and fuel consumption, supporting scalable relief operations even as the number of affected zones grows.

Society and Safety

Rapid, severity-aware dispatch helps relief agencies reach the most critical communities faster, reducing loss of life and suffering during a disaster.

Ethics and Privacy

Zone and population data may be sensitive. Access should be restricted to authorised relief personnel, and collected data should be used only for legitimate humanitarian purposes.

Professional Responsibility

The system must be tested carefully because an incorrect shortest-route calculation or an incorrect severity ranking could delay relief to a zone that urgently needs it.

8. Conclusion

The Intelligent Disaster-Relief Supply Chain Engine demonstrates the practical use of graphs, min-heaps and max-heap priority queues in humanitarian logistics. The graph models the road network, Dijkstra's algorithm with a min-heap computes the shortest delivery route to every zone, and the max-heap priority queue surfaces the most severe zones first for dispatch. Performance testing provides evidence of how priority-queue insertion behaves as the workload increases. The system can be extended in future with real-time traffic data, larger-scale zone networks, multiple warehouses and multithreaded route computation.

9. Student Reflection

This assignment helped us understand how data structures can be combined to solve a practical humanitarian-logistics problem. We learned the importance of graphs for modelling networks, min-heaps for efficient shortest-path computation, and max-heap priority queues for urgency-based ranking. We also learned that choosing the correct data structure depends on the operation that needs to be optimised.

With additional time and resources, we would extend the project to process live traffic and road-closure data, compare Dijkstra with A* search, test larger zone networks, add graphical performance analysis, and support multiple relief warehouses.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press, 2022.
- Sedgewick, R., & Wayne, K., Algorithms, Addison-Wesley, 2011.
- Weiss, M. A., Data Structures and Algorithm Analysis in C, Pearson, 2014.
- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B., Network Flows: Theory, Algorithms, and Applications, Prentice Hall, 1993.
- Knuth, D. E., The Art of Computer Programming, Volume 1: Fundamental Algorithms, Addison-Wesley, 1997.
- Black, P. E., Dictionary of Algorithms and Data Structures, National Institute of Standards and Technology (NIST).
- United Nations Office for Disaster Risk Reduction (UNDRR), Words into Action: Disaster Risk Reduction, 2021.
- Van Wassenhove, L. N., Humanitarian Aid Logistics: Supply Chain Management in High Gear, Journal of the Operational Research Society, 2006.
- NIST, Cybersecurity Framework (CSF) 2.0, National Institute of Standards and Technology, 2024.
- Sphere Association, The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response, 2018.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Assessment Criterion	Marks
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis:

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

